Neural networks unleashed: joint SPX/VIX calibration has never been faster

Joint calibration to the Standard & Poor's 500 (SPX) and Chicago Board Options Exchange (CBOE) Volatility Index (VIX) market data can be computationally burdensome, especially when the standard course of action for pricing volatility derivatives is nested Monte Carlo simulation, as is the case for the four-factor Markov path-dependent volatility model of Guyon and Lekeufack. A crucial boost to solving the joint problem was developed by Gazzani and Guyon, who trained a neural network to learn VIX as a random variable: the network replaces the inner simulation, and the pricing of VIX derivatives is reduced to a standard Monte Carlo computation. This is a step in the right direction but it is not sufficient. In this paper, Fabio Baschetti, Giacomo Bormetti and Pietro Rossi aim to further accelerate calibration by training neural networks to learn SPX implied volatilities, VIX futures and call option prices as a function of model parameters and contract specifications. As a result, pricing boils down to a matrix-vector product that can be calculated in real time, while calibration only takes a few seconds

Recent years have seen a deeper understanding of the role of path dependence in volatility modelling. Classical one-factor stochastic volatility (SV) models allow a latent process Y_t to implicitly depend on the historical price path of the asset through non-zero correlation with the spot price:

$$\begin{aligned}\frac{\mathrm{d}S_t}{S_t} &= \sigma_t\,\mathrm{d}W_t\\ \sigma_t &= f(t,Y_t)\\ Y_t &= Y_0 + \int_0^t \mu(t,u,Y_u)\,\mathrm{d}u\\ &\quad + \int_0^t v(t,u,Y_u)\left(\rho\frac{1}{f(u,Y_u)}\frac{\mathrm{d}S_u}{S_u} + \sqrt{1-\rho^2}\,\mathrm{d}W_u^{\perp}\right)\end{aligned}$$

with suitably specified μ and v. In particular, following standard classification:

- If $\rho = 0$, the model is strictly path-independent. Instantaneous volatility is a function of the Brownian path $(W_u^{\perp})_{0\leqslant u\leqslant t}$ alone, with absolutely no feedback from the price:

$$\sigma_t = \varphi(t,(\mathrm{d}W_u^{\perp})_{0\leqslant u\leqslant t})$$

- If $\rho \notin \{0,\pm 1\}$, the model is partially path-dependent. The asset price $(S_u)_{0\leqslant u\leqslant t}$ enters the volatility together with independent random shocks $(W_u^{\perp})_{0\leqslant u\leqslant t}$:

$$\sigma_t = \varphi\left(t,\left(\frac{\mathrm{d}S_u}{S_u}\right)_{0\leqslant u\leqslant t},(\mathrm{d}W_u^{\perp})_{0\leqslant u\leqslant t}\right)$$

- If $\rho = \pm 1$, the model is fully path-dependent. There is pure feedback from asset returns to volatility:

$$\sigma_t = \varphi\left(t,\left(\frac{\mathrm{d}S_u}{S_u}\right)_{0\leqslant u\leqslant t}\right)$$

Interestingly, the joint calibration of SV models to the Standard & Poor's 500 (SPX) and Chicago Board Options Exchange (CBOE) Volatility Index (VIX) seems to point towards the latter regime, with Guyon & Mustapha (2023) proving that a highly parameterised neural stochastic differential equation automatically selects degenerate (in time and space) correlation functions. Empirical evidence for path-dependence can be found in the works by Zumbach (2009, 2010) and Guyon (2014), among others.

The essential difference between SV and path-dependent volatility (PDV) models is that the path dependence produced by the former is complicated and implicit. Conversely, PDV models explicitly choose a precise functional form for the volatility function to describe, for example, how volatility levels evolve.

The four-factor Markov PDV model by Guyon & Lekeufack (2023) is the most famous (and promising) example within this class. Instantaneous volatility is a linear function of a weighted average of past asset returns R_1 and of the square root of a weighted average of past asset squared returns $\sqrt{R_2}$. The resulting model has a high predictive power for explaining realised volatility, and a simple modification allows for sensible pricing and (joint) calibration under the risk-neutral measure.

The main issue with the four-factor model is that pricing hinges on slow Monte Carlo integration. In particular, VIX derivatives require nested simulation, taking several minutes on a standard laptop. For every outer path, a round of inner simulation builds the VIX based on the terminal value of the state variables (here, the individual components of R_1 and R_2). This map between the state variables and the VIX was recently learned by parameterising the conditional expectation defining the VIX via a deep feed-forward neural network and training it for fast approximation of the inner simulation (Gazzani & Guyon 2025). While this is a huge step forward, the outer simulation remains. Our contribution consists in eliminating this last computationally burdensome step by learning the full map between the model parameters, the initial values of the volatility factors, strike and maturity, and the price of VIX derivatives.

Following our approach, a viable joint calibration can be performed entirely by neural networks. A series of matrix-vector products replace Monte Carlo integration, and calibration becomes extremely fast. In the following sections, we detail the learning procedure, assess the neural approximation quality and demonstrate that joint calibration can be performed in a few seconds (as opposed to about 15 minutes in Gazzani & Guyon (2025)).

This paper contributes to a flourishing literature that has, over the years, examined deep-learning approaches to the pricing and calibration of SV models. In particular, this process has mainly involved options on the SPX.

Copyright Infopro Digital Media Services

The work by Horvath *et al* (2021) is a prime example of using a grid-based method, while Baschetti *et al* (2024) revived the pointwise approach after realising that careful sampling of the training points dramatically improves learning. We build on their experience and extend our approach to VIX derivatives. To the best of our knowledge, Rosenbaum & Zhang (2022) have made the only attempt at genuinely deep joint calibration. However, they use a grid-based approach, which fails to reach market quotes (interpolation to/from the grid is needed) and only learns VIX option prices. Futures are never accounted for and are not included in the calibration process. We overcome both limitations by treating VIX options and futures with the same pointwise network. Thus, we provide a complete, flexible, fully neural approach to the joint calibration problem, which we apply to the Guyon-Lekeufack volatility model.

Notable machine learning applications in finance also include those by Huge & Savine (2020), McGhee (2021) and Ferguson & Green (2018).

The model

The paper is concerned with (an inflated version of) the original PDV model by Guyon & Lekeufack (2023):

$$\begin{aligned}\frac{\mathrm{d}S_t}{S_t} &= \sigma(R_{1,t}, R_{2,t})\,\mathrm{d}W_t\\ \sigma(R_{1,t}, R_{2,t}) &= \beta_0 + \beta_1 R_{1,t} + \beta_2\sqrt{R_{2,t}} + \beta_{1,2}R_{1,t}^2\mathbf{1}_{R_{1,t}\geq 0}\\ R_{1,t} &= \int_{-\infty}^{t} K_1(t-u)\frac{\mathrm{d}S_u}{S_u}\\ R_{2,t} &= \int_{-\infty}^{t} K_2(t-u)\left(\frac{\mathrm{d}S_u}{S_u}\right)^2\end{aligned}$$

Factors R_1 and R_2 are the weighted averages of past asset simple returns and of squared returns, respectively. They consequently account for trends and variability in the historical price path of the underlying. Leverage effects and volatility clustering are encapsulated, by construction, via the appropriate selection of the sign for coefficients β_1 and β_2. A (semi-)parabolic component pumps volatility in the presence of a positive price trend and gives the marginal distribution of the asset price sufficient mass in the right tail (see Gazzani & Guyon (2025) for a discussion about the role of coefficient $\beta_{1,2}$ on the shape of the SPX and VIX smiles), while β_0 sets a baseline volatility level that does not depend on the spot price.

The choice of a double exponential kernel (following the idea to mimic persistence via heterogeneous time scales):

$$K_{\theta,\lambda_0,\lambda_1}(\tau)\colon \tau \mapsto (1-\theta)\lambda_0 \mathrm{e}^{-\lambda_0\tau} + \theta\lambda_1\mathrm{e}^{-\lambda_1\tau} \quad \text{where } \theta\in[0,1],\ \lambda_0 > \lambda_1 > 0$$

effectively decomposes the factors as:

$$\begin{aligned}R_{1,t} &= (1-\theta_1)R_{1,0,t} + \theta_1 R_{1,1,t}\\ R_{2,t} &= (1-\theta_2)R_{2,0,t} + \theta_2 R_{2,1,t}\end{aligned}$$

and the individual elements:

$$\begin{aligned}R_{1,j,t} &= \mathrm{e}^{-\lambda_{1,j}t}R_{1,j,0} + \int_0^t \lambda_{1,j}\mathrm{e}^{-\lambda_{1,j}(t-u)}\sigma_u\,\mathrm{d}W_u, \quad j\in\{0,1\}\\ R_{2,j,t} &= \mathrm{e}^{-\lambda_{2,j}t}R_{2,j,0} + \int_0^t \lambda_{2,j}\mathrm{e}^{-\lambda_{2,j}(t-u)}\sigma_u^2\,\mathrm{d}u, \quad j\in\{0,1\}\end{aligned}$$

ultimately become Markovian.

We can easily recover their instantaneous dynamics:

$$\begin{aligned}\mathrm{d}R_{1,j,t} &= \lambda_{1,j}(\sigma(R_{1,t}, R_{2,t})\,\mathrm{d}W_t - R_{1,j,t}\,\mathrm{d}t), \quad j\in\{0,1\}\\ \mathrm{d}R_{2,j,t} &= \lambda_{2,j}(\sigma(R_{1,t}, R_{2,t})^2 - R_{2,j,t})\,\mathrm{d}t, \quad j\in\{0,1\}\end{aligned}$$

The model parameters have the following straightforward interpretation.

- $\beta_0 > 0$ is the baseline volatility, and $\beta_1 < 0$ and $\beta_2 > 0$ are the linear coefficients of the trend and the (square root of) the variability features, respectively, while $\beta_{1,2} \geq 0$ is the parabolic term reproducing the probability mass on the right wing of SPX smiles.
- $\lambda_{1,0} > 0$ captures the dependence of R_1 on recent returns: the higher the $\lambda_{1,0}$, the greater the weights of recent returns.
- $\lambda_{1,1} < \lambda_{1,0}$ captures the dependence of R_1 on older returns: the smaller the $\lambda_{1,1}$, the greater the weights of older returns.
- $\lambda_{2,0} > 0$ captures the dependence of R_2 on recent squared returns: the higher the $\lambda_{2,0}$, the greater the weights of recent squared returns.
- $\lambda_{2,1} < \lambda_{2,0}$ captures the dependence of R_2 on older squared returns: the smaller the $\lambda_{2,1}$, the greater the weights of older squared returns.
- θ_1 and θ_2 mix the dependence of R_1 and R_2 on recent and older returns – and squared returns.

Pricing VIX derivatives

The VIX follows the market definition below:

$$\mathrm{VIX}_T^2 = \mathrm{Price}_T\left[-\frac{2}{\Delta}\log\left(\frac{S_{T+\Delta}}{F_T^{T+\Delta}}\right)\right] \tag{1}$$

where F_t^u denotes the time-t price of the SPX futures expiring at $u \geq t$, and $\Delta = \frac{30}{365}$ is a 30-day horizon.

The model counterpart of (1) is given by:

$$\mathrm{VIX}_T^2 = \mathbb{E}\left[-\frac{2}{\Delta}\log\left(\frac{S_{T+\Delta}}{F_T^{T+\Delta}}\right)\,\middle|\,\mathbb{F}_T\right] \tag{2}$$

If there are no jumps in the underlying S, then (2) reduces to:

$$\begin{aligned}\mathrm{VIX}_T^2 &= \mathbb{E}\left[\frac{1}{\Delta}\int_T^{T+\Delta}\sigma_u^2\,\mathrm{d}u\,\middle|\,\mathbb{F}_T\right]\\ &= \frac{1}{\Delta}\int_T^{T+\Delta}\mathbb{E}[\sigma_u^2\mid\mathbb{F}_T]\,\mathrm{d}u\\ &= \frac{1}{\Delta}\int_T^{T+\Delta}\xi_T^u\,\mathrm{d}u\end{aligned} \tag{3}$$

via a straightforward application of Itô's formula. Normalisation by Δ is now justified to make the VIX an average of the expected future variance on the SPX.

Market participants actively trade VIX options and futures to manage volatility directly:

$$\begin{aligned}F_{\mathrm{VIX}}(t,T) &= \mathbb{E}[\mathrm{VIX}_T\mid\mathbb{F}_t]\\ C_{\mathrm{VIX}}(t,T,K) &= \mathbb{E}[(\mathrm{VIX}_T - K)^+\mid\mathbb{F}_t]\end{aligned}$$

Here, VIX_T is a conditional expectation, and the price of VIX derivatives consequently appears as the expectation of a conditional expectation. Nested Monte Carlo simulation is an unbiased but very slow solution for this kind of problem. The good news is that we can leverage least squares techniques to improve performance dramatically.

Copyright: Infopro Digital Media Services

When talking about least squares Monte Carlo, we mean an acceleration for nested simulation based on regression techniques. The idea is to perform the inner simulation only on a subsample of the outer paths and generalise via statistical methods. This trick significantly boosts the pricing of VIX derivatives but is still far from being sufficiently fast for real-time calibration purposes. Joint calibration remains clearly beyond the scope of a standard laptop. Nonetheless, the boost makes sample generation for training a neural network (which we do in the following) affordable.

Methodology

Calibration to both the SPX and the VIX market requires two separate networks. We discuss each of them individually. We leverage the pointwise approach (see Baschetti *et al* (2024) for details), which can effectively be used for pricing without any need for interpolation/extrapolation over moneyness and time-to-maturity.

■ **Learning SPX implied volatilities** Neural network pricing of SPX options under a given model $\mathcal{M}$ hinges on the mapping below:

$$\Phi_w^{\mathcal{M},\mathrm{SPX}}\colon (\boldsymbol{\theta}, T, K) \mapsto \sigma_{\mathrm{BS}}^{\mathcal{M},\mathrm{SPX}} \tag{4}$$

where w denotes the set of weights that make up the approximation to the (true) model pricing function. The model parameters $\boldsymbol{\theta}$ and contract specifications (T, K) are given as an input, and the network returns a single point in implied volatility.

Pointwise methods are traditionally opposed to grid-based neural network techniques, where the target is a whole set of volatilities on prespecified strike-maturity pairs. As such, one can only price selected T and K by a grid-based network and reach actual market options via interpolation and extrapolation. In this respect, a pointwise network is much more flexible than a grid-based one. This fact guides our choice of method. Still, we train the network on points from randomly generated grids and refer the reader to Baschetti *et al* (2024) for a discussion of the benefits associated with this choice.

The decision to learn implied volatilities directly is a standard one, usually motivated by the fact that volatilities typically vary in a small range with no extreme values, and learning does not need any rescaling of the target (as opposed to very small prices for deep out-of-the-money options). We will calibrate on implied volatilities as well; they are in fact much more sensible to use than prices when trying to reach the far wings of the smile.

The first step is sample generation. We draw parameter combinations and contract specifications and associate each with the Monte Carlo option price.

For the readers' convenience we list the model parameters here:

$$\boldsymbol{\theta} = (\underbrace{\beta_0, \beta_1, \beta_2, \beta_{1,2}, \lambda_{1,0}, \lambda_{1,1}, \theta_1, \lambda_{2,0}, \lambda_{2,1}, \theta_2}_{\boldsymbol{\theta}^{\mathcal{M}}},$$
$$\underbrace{R_{1,0,0}, R_{1,1,0}, R_{2,0,0}, R_{2,1,0}}_{\boldsymbol{\theta}^{R}})$$

and we specify their bounds in table A.

We make sure that:

$$\lambda_{1,0} > \lambda_{1,1}$$
$$\lambda_{2,0} > \lambda_{2,1}$$

so that the first factor is the one that mean-reverts faster. This is important in order to prevent the neural network from learning the symmetry due to the exchange of $(\lambda_{n,0}, 1-\theta_n)$ with $(\lambda_{n,1}, \theta_n)$. We impose the same hierarchy of λs throughout calibration via linear inequality constraints.

A. Model parameter bounds during the learning phase

Parameter	Bounds
β_0	[0, 0.85]
β_1	[−0.30, −0.10]
β_2	[0.35, 0.95]
$\beta_{1,2}$	[0.05, 0.40]
$\lambda_{1,0}$	[10, 65]
$\lambda_{1,1}$	[0, 35]
θ_1	[0, 1]
$\lambda_{2,0}$	[0, 50]
$\lambda_{2,1}$	[0, 15]
θ_2	[0, 1]
$R_{1,0,0}$	[−1.62, 0.88]
$R_{1,1,0}$	[−1.05, 0.71]
$R_{2,0,0}$	[0, 0.11]
$R_{2,1,0}$	[0, 0.11]

Also, and most importantly, sampling is not uniform in parameter space. The initial values of the factors are inherently linked to the λ values via the past history of the index. Effectively, this means that:

$$R_{n,j,0} = \lambda_{n,j} \sum_{i=0}^{T-2} \mathrm{e}^{-\lambda_{n,j}\frac{i}{252}} \left(\frac{S_{-i}}{S_{-i-1}} - 1\right)^n$$
$$n = \{1, 2\},\ j = \{0, 1\}$$

where $T = 1{,}008$ (ie, we have a price history of four years), and S_0 is the index level from a date randomly sampled within the period from January 1, 2009 to December 31, 2023.

Finally, only a fraction of the parameter combinations we have drawn enters the training sample, as many correspond to unrealistic surfaces. We perform the selection automatically by imposing minimal shape and level constraints. However, in order to ensure the neural network properly explores the entire hypercube defined by table A, we also retain a portion of parameters corresponding to unrealistic shapes.

Each $\boldsymbol{\theta}$ in the training sample is associated with a random grid of times-to-maturity and strikes. More specifically, we sample 11 expirations uniformly at random from the time subintervals in $[T_{\min}, T_{\max}]$ below:

$$[T_{\min}, T_{\max}] = [6/365, 1/12) \cup [1/12, 2/12) \cup [2/12, 3/12)$$
$$\cup [3/12, 4/12) \cup [4/12, 5/12) \cup [5/12, 6/12)$$
$$\cup [6/12, 8/12) \cup [8/12, 10/12) \cup [10/12, 11/12)$$
$$\cup [11/12, 1) \cup [1, 13/12]$$

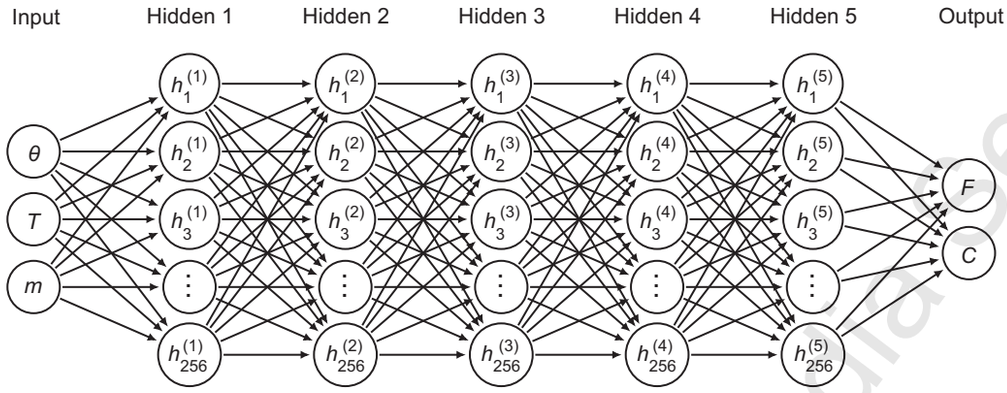
The strikes are also random, but they extend around the at the money according to the following rule:

$$[K_{\min}(T), K_{\max}(T)] = [S_0(1 - l\sqrt{T}), S_0(1 + u\sqrt{T})]$$
$$l = 0.55,\ u = 0.30$$

■ **Learning VIX derivative pricing** Regarding VIX derivatives, we employ a single network to learn both futures and option prices consistently. The fact that the target is now a price is extremely natural for several reasons. First, there is no such thing on the market as an implied volatility for futures. The

Copyright: Infopro Digital Media Services

1 A neural network architecture for learning VIX derivatives



The input layer is $(p+2)$ dimensional, where $p = 14$ is the number of model parameters. The output layer is two dimensional: F is the futures price, C is the call option price

implied volatility of a VIX option depends on the VIX future with the same expiry, while the price of a VIX option does not. Second, learning implied volatilities directly may be affected by two sources of statistical uncertainty, one associated with the Monte Carlo computation of the option price and the other with the computation of the futures price.

Again, we use the same technology of a random-grid pointwise approach as described above, but we do not enforce any shape constraints on the VIX smiles. Our time bucket is as follows:

$$[T_{\min}, T_{\max}] = [6/365, 18/365) \cup [18/365, 30/365)$$

and we draw 20 strikes per smile. Strikes are quoted in terms of the futures price, and they vary in the interval

$$\frac{K}{F} = [0.82, 2.36]$$

irrespective of the expiration. As a consolidated practice, we distinguish three levels of moneyness and take

- four strikes in $[0.82, 1.00]$,
- seven strikes in $[1.00, 1.40]$ and
- nine strikes in $[1.40, 2.36]$.

Generating the training sample via nested Monte Carlo simulation is challenging. Indeed, even though the procedure can be performed offline, the number of configurations to explore is large, translating into a considerable number of computation hours. To render the procedure affordable, we must significantly reduce the number of calls to the inner loop. The key strategy is to compute the VIX via Monte Carlo simulation over the inner trajectories only for a relatively small subsample of outer paths. We can then project the subsample of VIX points onto polynomials in the state variables via penalised least squares and subsequently generalise the polynomial approximation to the whole sample of outer paths. We suggest a ridge penalisation to preserve the closed-form expression of the regression coefficients and speed up the procedure. Penalising the regression is essential to limit the variability of the linear regression coefficients from sample to sample and prevent overfitting the data. We use regressors $(R_{1,0}, R_{1,1}, R_{2,0}, R_{2,1}, \sigma)$ and $d = 3$ as the highest power for the monomials.

B. Summary statistics from the distribution of the error on the VIX call and VIX future as measured from the whole test set

	Call option price	Future
MAE	7.2049e-05	2.5981e-04
$q_{0.95}^{\lvert err\rvert}$	2.1556e-04	6.9774e-04

We computed the nested Monte Carlo benchmarks, generating 2^{18} outer paths and drawing 2^{13} inner trajectories for each. In our efficient procedure, we called the inner loop only 2^{13} times and generalised the VIX to 2^{18} outer paths using the regression formula.

The rows in the training sample were given by:

$$(\boldsymbol{\theta}, T_1, T_2, m_1^1, \ldots, m_1^{20}, m_2^1, \ldots, m_2^{20}, F_1, F_2, C_1^1, \ldots, C_1^{20}, C_2^1, \ldots, C_2^{20}).$$

The samples were flattened for pointwise training and passed to a feed-forward neural network whose structure is described in figure 1.

We could alternatively have broken the VIX problem into two subnetworks, one for VIX futures and one for VIX options (prices or volatilities). We experimented with the alternative approaches and found that the above strategy is the most accurate and robust.

To help the reader gauge the accuracy of the network, table B lists a few summary statistics from the test set.

The objective function

Following Gazzani & Guyon (2025), we optimise only over the parameters $\boldsymbol{\theta}^{\mathcal{M}} \in \mathbb{R}^{10}$. At every iteration in the calibration loop, and for every $n = \{1, 2\}$ and $j = \{0, 1\}$, we evaluate $R_{n,j,0}$ based on the current value of $\lambda_{n,j}$ and the past history of the SPX (four years from the calibration date) to build $\boldsymbol{\theta}^{R}$. We stack:

$$\boldsymbol{\theta} = (\boldsymbol{\theta}^{\mathcal{M}}, \boldsymbol{\theta}^{R})$$

and pass it to the neural network(s) for evaluation of the loss and its gradient. We take one step and repeat until convergence. Then, strictly speaking, $\boldsymbol{\theta}^{R}$ is not calibrated, but it is actually implied by the optimal $\boldsymbol{\theta}^{\mathcal{M}}$. This point is precisely where path-dependence enters the scene. The idea is that we should always condition on the asset price path as observed at a certain point in time

Copyright Infopro Digital Web Services

C. Joint SPX-VIX calibration of the four-factor Markov PDV model as of October 21, 2009

β_0	β_1	β_2	$\beta_{1,2}$	$\lambda_{1,0}$	$\lambda_{1,1}$	θ_1	$\lambda_{2,0}$	$\lambda_{2,1}$	θ_2
0.0850	−0.2637	0.6614	0.2164	50.05	7.52	0.8888	5.70	0.29	0.9750

The model parameters are as in table A. The initial values of the factors are $R_{1,0,0} = 0.0163$, $R_{1,1,0} = 0.4337$, $R_{2,0,0} = 0.0398$, $R_{2,1,0} = 0.0581$. The futures price from the neural network, $F_{\text{nn}} = 0.2457$. The futures price from nested Monte Carlo, $F_{\text{MC}} = 0.2461$

when calibration is performed. Initialisation is, by construction, consistent with the market price history under the empirical measure, while the model describes its risk-neutral evolution.

As for the loss function, for the joint problem we must account for

- the contribution from the SPX smiles,
- the contribution from VIX futures and
- the contribution from the VIX smiles.

We opt for the following:

$$
\begin{aligned}
L^J = {} & w_{v\text{SPX}} \frac{1}{N_T^{\text{SPX}}} \\
& \times \sum_{i=1}^{N_T^{\text{SPX}}} \left[\frac{1}{N_{K(T_i^{\text{SPX}})}} \right. \\
& \qquad \left. \times \sum_{l=1}^{N_{K(T_i^{\text{SPX}})}} \left(\frac{\sigma_{\text{mdl}}^{\text{SPX}}(T_i^{\text{SPX}}, K_l(T_i^{\text{SPX}}))}{\sigma_{\text{mkt}}^{\text{SPX}}(T_i^{\text{SPX}}, K_l(T_i^{\text{SPX}}))} - 1 \right)^2 \right] \\
& + w_{f\text{VIX}} \frac{1}{N_T^{\text{VIX}}} \sum_{i=1}^{N_T^{\text{VIX}}} \left(\frac{F_{\text{mdl}}^{\text{VIX}}(T_i^{\text{VIX}})}{F_{\text{mkt}}^{\text{VIX}}(T_i^{\text{VIX}})} - 1 \right)^2 + w_{v\text{VIX}} \frac{1}{N_T^{\text{VIX}}} \\
& \times \sum_{i=1}^{N_T^{\text{VIX}}} \left[\frac{1}{N_{K(T_i^{\text{VIX}})}} \right. \\
& \qquad \left. \times \sum_{l=1}^{N_{K(T_i^{\text{VIX}})}} \left(\frac{\sigma_{\text{mdl}}^{\text{VIX}}(T_i^{\text{VIX}}, F_{\text{mdl}}^{\text{VIX}}(T_i^{\text{VIX}}), K_l(T_i^{\text{VIX}}))}{\sigma_{\text{mkt}}^{\text{VIX}}(T_i^{\text{VIX}}, F_{\text{mkt}}^{\text{VIX}}(T_i^{\text{VIX}}), K_l(T_i^{\text{VIX}}))} - 1 \right)^2 \right]
\end{aligned}
\tag{5}
$$

where:

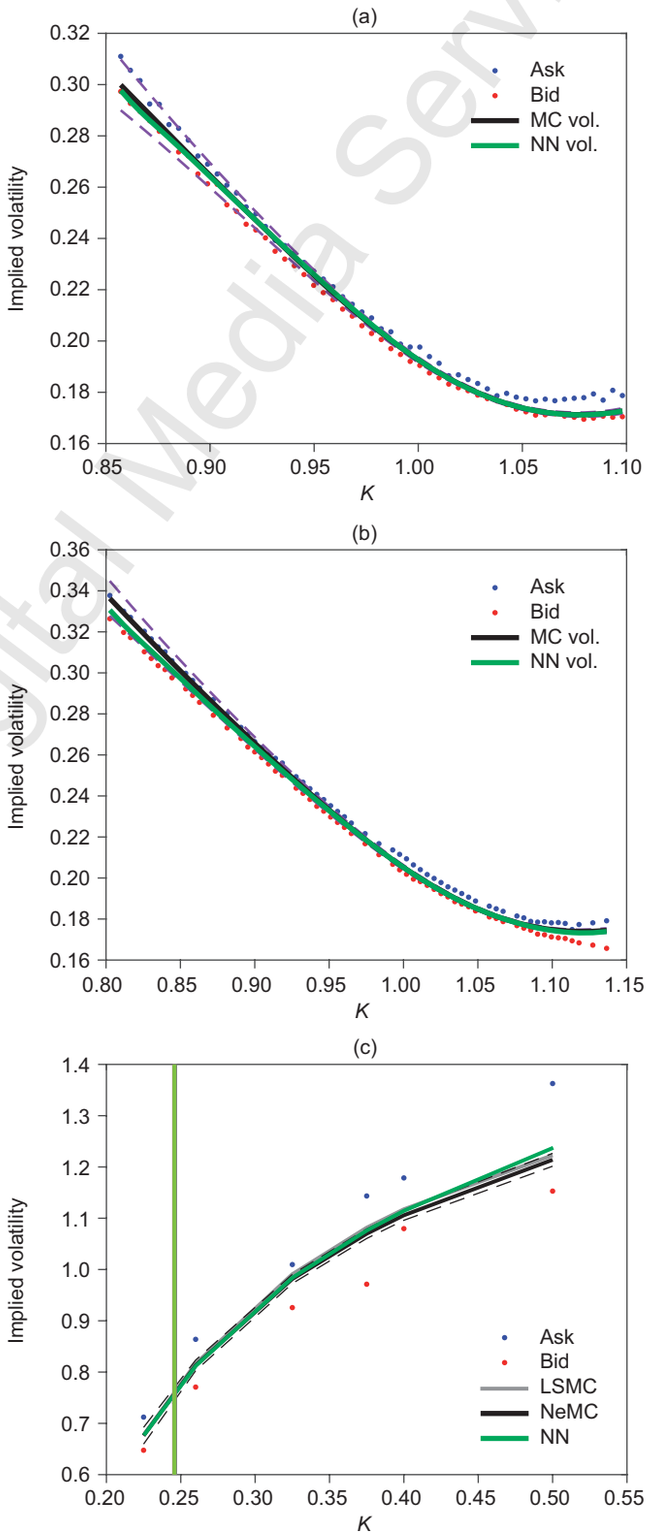
- $w_{v\text{SPX}}, w_{f\text{VIX}}, w_{v\text{VIX}} \in \mathbb{R}$ are the weights of the three contributions above;
- $N_T^{\text{SPX/VIX}}$ is the number of SPX/VIX maturities; and
- $N_{K(T_\cdot^{\text{SPX/VIX}})}$ is the number of strikes for a given SPX/VIX maturity.

Equation (5) is very similar to the one proposed by Guyon & Mustapha (2023). The main difference is that Guyon & Mustapha calibrate on VIX call option prices directly to avoid recovering the implied volatilities from a noisy futures price. Their procedure is entirely Monte Carlo based, and the noise in the futures price may severely affect the inversion from a noisy option price. Because we have learned futures and option prices consistently from a Monte Carlo procedure that strongly leverages the efficiency provided by the least squares technique, we can safely proceed with calculating the implied volatilities.

Neural net joint calibration in action

Similarly to Gazzani & Guyon (2025), we only look at the first (shortest) triangle (ie, two SPX smiles and one VIX slice). In fact, on both markets, short-dated options are by far the most liquid.

2 Joint SPX-VIX calibration of the four-factor Markov PDV model as of October 21, 2009



The neural network (NN) fit is shown by the solid green line. Monte Carlo benchmarks (95% confidence bands) are shown by the solid black (dashed purple) line. (a) $T = 31$ days. (b) $T = 59$ days. (c) $T = 28$ days

Copyright Infopro Digital Media Services

We calibrate using our neural approximations and validate the fits via comparison with (nested) Monte Carlo. Table C stores optimal model parameters as calibrated on October 21, 2009. Figure 2 provides the resulting fit to the market. The VIX benchmark comes with $N_{\text{sim\_out}} = 2^{18}$ outer paths and $N_{\text{sim\_inn}} = 2^{13}$ inner paths. We also superimpose the least squares Monte Carlo (LSMC) fit and demonstrate not only coherence with the neural network fit (confirming once more that the network properly learned the least squares pricing) but also that it is very close to the nested simulation benchmark. The latter fact supports the effectiveness of the polynomial regression.

Conclusions

We provided a deep neural network pricer for SPX options and VIX derivatives in the Guyon-Lekeufack volatility model that can be used for real-time calibration to market data. This addresses a fundamental problem for practitioners that was not entirely solved by Gazzani & Guyon (2025). In summary, the main contributions of this paper are as follows.

- We accelerated the generation of VIX samples needing nested simulation by leveraging least squares techniques.
- We rendered joint calibration fully neural by building two separate networks, one for SPX implied volatilities and one for VIX futures and options prices.
- We showed with a real-data calibration that the procedure outlined above is extremely fast and provides accurate results (specifically, our joint calibration is hundreds of times faster (from 15 minutes to a few seconds) than that described by Gazzani & Guyon (2025)). ■

Fabio Baschetti is doing a postdoctoral researcher in the Department of Economics at the University of Verona. Giacomo Bormetti is a full professor of mathematical methods for economics and finance in the Department of Economics and Management at the University of Pavia. Pietro Rossi is an adjunct professor of computational finance in the Department of Statistical Sciences 'Paolo Fortunati' at the University of Bologna and is also a senior advisor to Prometeia SpA. They acknowledge the CINECA award under the ISCRA initiative, for the availability of high-performance computing resources and support. They warmly thank Guido Gazzani for fruitful discussions, as well as the participants at the Workshop on Numerical Methods for Finance and Insurance, Milan, 2025, and the Financial Engineering Workshop at the Bayes Business School, City St George's, University of London for insightful questions.
Email: fabio.baschetti@univr.it, giacomo.bormetti@unipv.it, pietro.rossi@prometeia.com.

REFERENCES

Baschetti F, G Bormetti and P Rossi, 2024
Deep calibration with random grids
Quantitative Finance 24(9), pages 1,263–1,285

Ferguson R and A Green, 2018
Deeply learning derivatives
Preprint, arXiv:1809.02233

Gazzani G and J Guyon, 2025
Pricing and calibration in the 4-factor path-dependent volatility model
Quantitative Finance 25(3), pages 471–489

Guyon J, 2014
Path-dependent volatility
Risk September, www.risk.net/2372193

Guyon J and J Lekeufack, 2023
Volatility is (mostly) path-dependent
Quantitative Finance 23(9), pages 1,221–1,258

Guyon J and S Mustapha, 2023
Neural joint S&P 500/VIX smile calibration
Risk November, www.risk.net/7958175

Horvath B, A Muguruza and M Tomas, 2021
Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models
Quantitative Finance 21(1), pages 11–27

Huge B and A Savine, 2020
Differential machine learning: the shape of things to come
Risk October, www.risk.net/7688441

McGhee W, 2021
An artificial neural network representation of the SABR stochastic volatility model
Journal of Computational Finance 25(2), pages 1–27

Rosenbaum M and J Zhang, 2022
Deep calibration of the quadratic rough Heston model
Risk October, www.risk.net/7954656

Zumbach G, 2009
Time reversal invariance in finance
Quantitative Finance 9(5), pages 505–515

Zumbach G, 2010
Volatility conditional on price trends
Quantitative Finance 10(4), pages 431–442

GUIDELINES FOR THE SUBMISSION OF TECHNICAL ARTICLES

Risk.net welcomes the submission of technical articles on topics relevant to our readership. Core areas include market and credit risk measurement and management, the pricing and hedging of derivatives and/or structured securities, the theoretical modelling of markets and portfolios, and the modelling of energy and commodity markets. This list is not exhaustive.

The most important publication criteria are originality, exclusivity and relevance. Given Risk.net technical articles are shorter than those in dedicated academic journals, clarity of exposition is another yardstick for publication. Once received by the quant finance editor and his team, submissions are logged and checked against these criteria. Articles that fail to meet the criteria are rejected at this stage.

Articles are then sent to one or more anonymous referees for peer review. Our referees are drawn from the research groups, risk management departments and trading desks of major financial institutions, in addition to academia. Many have already published articles in Risk.net. Authors should allow four to eight weeks for the refereeing process. Depending on the feedback from referees, the author may attempt to revise the manuscript. Based on this process, the quant finance editor makes a decision to reject or accept the submitted article. His decision is final.

Submissions should be sent to the technical team (technical@infopro-digital.com).

PDF is the preferred format. We will need the LaTeX code, including the BBL file, and charts in EPS or XLS format. Word files are also accepted.

The maximum recommended length for articles is 4,500 words, with some allowance for charts and formulas. We reserve the right to cut accepted articles to satisfy production considerations.

Copyright: Infopro Digital Media Services